Universidad Autónoma de Baja California

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Compilación de
Práctica 2#

Asignatura

Embebidos

Presentan:

Diego Bejarano Bedolla

Profesor :

Dr. Adolfo Heriberto Ruelas Puente

Mexicali, B.C. México
Febrero de 2026

Índice general

Índice de figuras **1**

1. Práctica 2 **2**
1.1. Introducción 2
1.2. Desarrollo 3
1.2.1. Lista de codigo fuente 6
1.3. Resultados 6
1.4. Conclusiones 8

Bibliografía **9**

Índice de figuras

1.1. Chip de un FTDI 3
1.2. ATmega328p 3
1.3. Código ISP 4
1.4. Configuración de la placa 5
1.5. FTDI 5
1.6. Diagrama seguido 6
1.8. Led prendido con arduino minomo 7
1.7. Armado del Arduino mínimo 7

Índice de tablas

1

Práctica 2

Diego Bejarano Bedolla

Resumen: Esta práctica busca comprender el funcionamiento de los sistemas embebidos, las necesidades que tienen que tener, es por esto que nos enfocaremos en el microcontrolador ATmega328P, el mismo que utiliza Arduino Uno, por lo que haremos nuestra propia versión de este.

1.1. Introducción

Para esta práctica realizaremos la versión mínima de Arduino Uno, esto es básicamente lo mínimo e indispensable que necesita el microcontrolador del Arduino Uno, el ATmega328p, para funcionar y poder cargar código a este. Para realizar esto, es necesario saber qué componentes ocupa el microcontrolador ATmega328p ya que esto no funciona por sí solo. Por lo que el objetivo principal de esta práctica es el de armar en protoboard el modo mínimo de Arduino Uno, el cual nos permita cargar código como si fuera una placa más.

Algo muy importante que necesitaremos es un convertidor de USB a UART, mejor conocido como FTDI, este nos ayuda a programar el arduino minimo para poder hacer la comunicación entre la computadora y el microcontrolador ATmega328p. Los controladores USB FTDI se utilizan principalmente para permitir la comunicación entre un ordenador y dispositivos USB que incorporan chips convertidores USB a serie de FTDI. Los chips convertidores USB-serie de FTDI se utilizan ampliamente en diversos dispositivos y aplicaciones electrónicas, como equipos industriales, electrónica de consumo y sistemas integrados. Estos chips proporcionan un puente entre las interfaces USB y serie, permitiendo que dispositivos sin puertos USB integrados se comuniquen con una computadora a través de USB.[2]

Figura 1.1: Chip de un FTDI

Otro concepto que tenemos que conocer es el del microcontrolador ATmega328p, este es un microcontrolador que ya hemos utilizado en distintas materias y en este tenemos bastante experiencia, por lo que ya sabemos cómo trabajar con este. El atmega328P es un CI (circuito integrado) de alto rendimiento que esta basado en un microcontrolador RISC (computador conjunto de instrucciones reducidas), con una memoria de 32KB flash con capacidad de leer entradas mientras esté ejecutando alguna acción en la parte de las salidas. Este dispositivo es uno de los microcontroladores básicos más completos que existe en la actualidad, ya que este posee una memoria EEPROM de 1KB, una memoria SRAM de 2KB, varios periféricos de comunicación (USART), (ICSP), (I2C), entradas digitales, entradas analógicas, salidas digitales, salidas PWM.[1]

Figura 1.2: ATmega328p

1.2. Desarrollo

El primer paso es comprender qué es lo que necesita el microcontrolador ATmega328p para funcionar, qué necesita internamente para que, primero, pueda prender y, segundo, que se le pueda cargar código. Ya que para hacer funcionar el modo minimo del Arduino Uno se puede dividir en esas dos partes, la parte física que son los componentes y conexiones necesarisas para hacer funcionar el ATmega328p

y la segunda parte el como cargar código.

Para que el Atmega328P funcione, lo mínimo necesario es conectar sus pines 7 y 20 a VCC (5V) y los 8 y 22 a tierra. Aunque el Atmega328P puede funcionar con un oscilador de reloj interno de 8MHz, sin bootloader no será posible activarlo, de modo que se debe usar un cristal externo de 16MHz, que se conecta a los pines 9 y 10, con sus dos capacitores de desacople (dos cerámicos de 22pF). El programa que escribirá el bootloader requiere mínimas conexiones: una al pin de Reset (con la resistencia de 10K) y tres a los pines 13, 12 y 11.

El siguiente paso es cargar el bootloader a nuestro microcontrolador, para esto hay que cargar un código que ya viene en el IDE de Arduino llamado ISP, una vez cargado este código también hay que configurar para que nuestro Arduino Uno con el que estamos quemando nuestro microcontrolador ATmega328p funcione como programador. Ya por último hay que seleccionar la placa Arduino nano y seleccionar el procesador ATmega328p para que todo funcione correctamente.

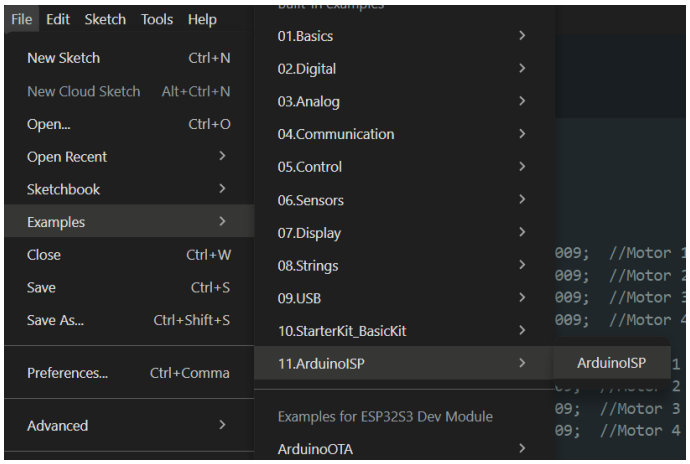

Figura 1.3: Código ISP

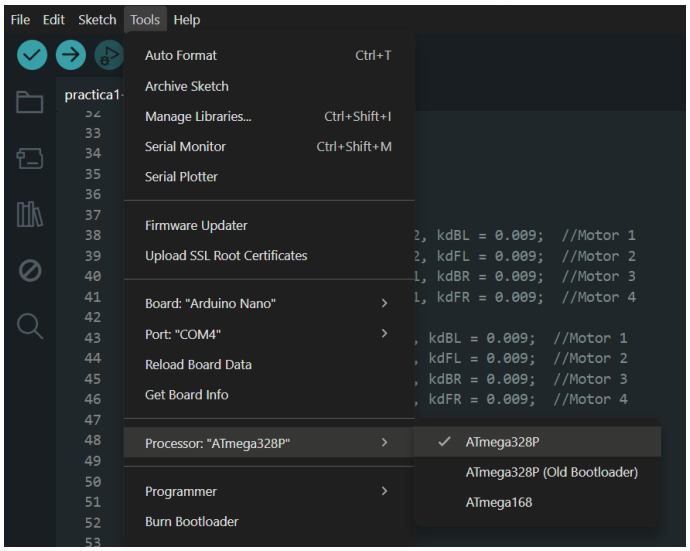

Figura 1.4: Configuración de la placa

Una vez hecho esto nuestro microcontroldor ATmega328p que usamos para nuestro Arduino Uno minimo queda listo con el bootloader, lo que quiere decir que ya podemos cargar código y usarlo como si fuera otro Arduino más, pero antes de hacer todo esto requerimos de otro componente, un FTDI, este es un convertidor de USB a serie UART (TTL) que permite a una computadora comunicarse con microcontroladores y dispositivos electrónicos embebidos. Este es esencial para poder cargard código desde nuestra computadora hasta nuestro Arduino Uno mínimo.

Figura 1.5: FTDI

A modo de diagrama, estas fueron las conexiones realizadas para la práctica, en donde los valores de los capacitores y resistencias fueron los mencionados con anterioridad, por todo lo demás, se hizo la misma conexión del microcontrolador ATmega328p y del FTDI.

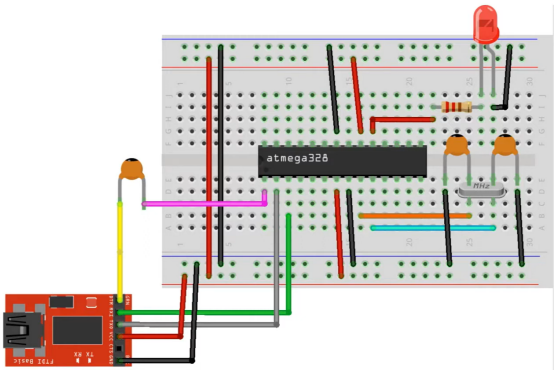

Figura 1.6: Diagrama seguido

1.2.1. Lista de codigo fuente

Para probar que nuestras conexiones y la configuración hechas fueron las correctas, se cargó este código como prueba, para esto se conectó en el pin 19 del microcontrolador un led rojo, este pin del ATmega328p corresponde con el pin 13 del Arduino. El código solamente enciende y apaga el led con un delay, si realizamos todas las conexiones correctamente, este código se puede cargar al microcontrolador y el LED parpadearía cada segundo.

```
const int led = 13;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
}
```

1.3. Resultados

Como resultado, se obtuvo que nuestro Arduino Uno mínimo puede ser cargado con código, como si fuera un Arduino convencional, pero al ser el modo mínimo, tiene algunas limitaciones, principalmente relacionadas con la ausencia de componentes auxiliares como el convertidor USB–serial integrado, el regulador de voltaje y ciertos circuitos de protección. Esto implica que para programarlo es necesario utilizar un programador externo o un adaptador como el FTDI que mencionamos anteriormente, además de cuidar la alimentación eléctrica para no dañar el microcontrolador.

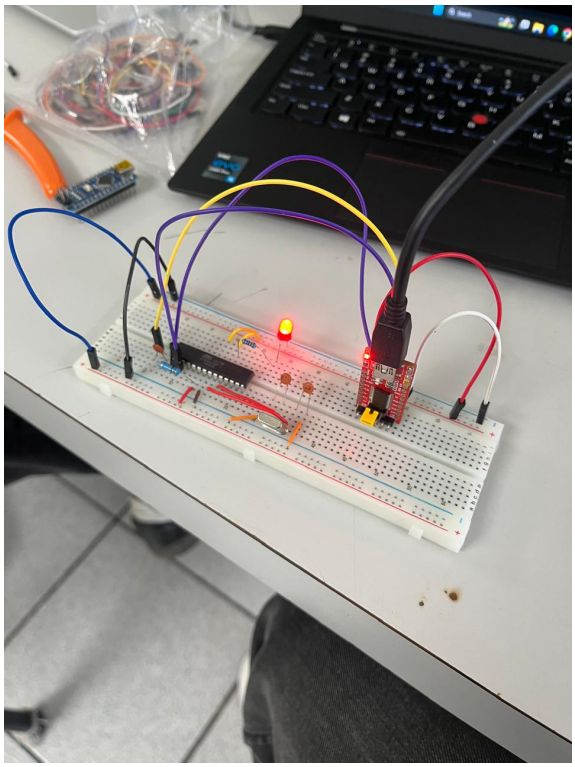

Figura 1.8: Led prendido con arduino minomo

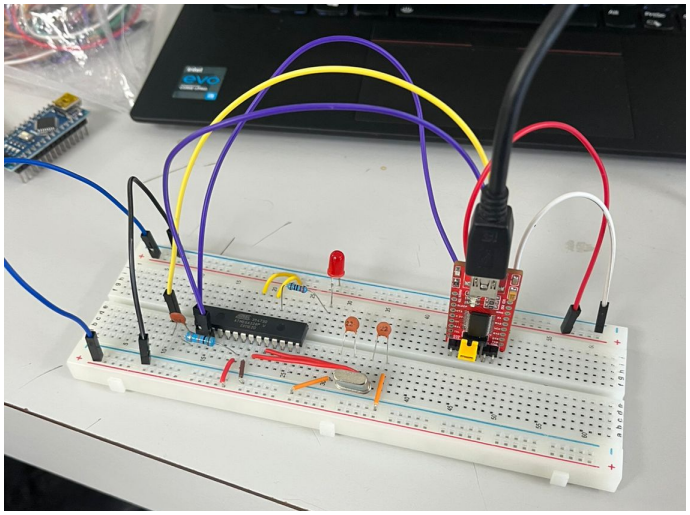

Figura 1.7: Armado del Arduino mínimo

A pesar de estas limitaciones, se comprobó que el microcontrolador funciona correctamente ejecutando el código cargado, respondiendo a las entradas y generando las salidas esperadas. Esto demuestra que el funcionamiento esencial del Arduino reside en el microcontrolador y que es posible implementar sistemas embebidos más compactos y económicos cuando se conocen bien los requerimientos del hardware. En general, esta experiencia permitió comprender mejor la estructura interna del Arduino y la diferencia entre una placa comercial completa y su configuración mínima de operación.

1.4. Conclusiones

En esta práctica se logró comprender de manera más profunda cómo funciona realmente un sistema embebido y qué componentes son indispensables para que un microcontrolador opere correctamente. Al construir el modo mínimo del Arduino Uno utilizando el ATmega328P, se comprobó que la placa comercial no es más que una integración de distintos módulos que facilitan su uso, pero que el funcionamiento esencial depende únicamente del microcontrolador, la alimentación adecuada, el sistema de reloj y el bootloader.

El proceso de quemar el bootloader y utilizar un FTDI para cargar código permitió entender cómo se establece la comunicación entre la computadora y el microcontrolador, así como la importancia de la correcta configuración en el IDE de Arduino. Además, al probar el sistema con un programa sencillo para hacer parpadear un LED, se verificó que las conexiones y configuraciones realizadas fueron correctas.

En general, esta práctica permitió reforzar conocimientos tanto de hardware como de software, comprendiendo mejor la estructura interna del Arduino y la lógica detrás de su funcionamiento. También mostró que es posible diseñar sistemas más simples y económicos cuando se conocen los requerimientos mínimos del microcontrolador, lo cual es fundamental para el desarrollo de proyectos embebidos más avanzados en el futuro.

Bibliografía

[1] ElectroAllWeb. Funcionamiento de todos los pines del microcontrolador atmega328p. Sitio web, 2020. Accedido el 26 de febrero de 2026.

[2] PCM Cable Co., Ltd. What is ftdi usb driver? Sitio web, 2023. Accedido el 26 de febrero de 2026.